



Міністерство освіти та науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря
Сікорського» Факультет інформатики та обчислювальної
техніки
Кафедра інформатики і програмної інженерії

Звіт
з дисципліни «Компоненти програмної інженерії»
Лабораторна робота №4
"Event-driven архітектура"

Виконав:

Студент II курсу

гр. ІІІ-33

Соколов О. В.

Перевірив:

Зарічковий О. А.

Лабораторні робота №4.

Event-driven архітектура

Тема: Event-driven архітектура.

Мета: Застосувати event-driven та serverless архітектуру для проектування складних систем.

Постановка задачі:

1. Вкажіть варіанти використання (use case) у вашій системі, які потребують event-driven архітектури.

2. Вкажіть, які саме патерни event-driven архітектури використовуються.

3. Вкажіть, які інструменти та технології використовуються та чому.

4. Оновіть діаграми.

5. Event-driven system PoC чи Stream processing PoC.

1. Якщо до вашої теми не підходить ідея stream processing, то необхідно зробити PoC для event-driven частини - для одного сценарію.

2. Якщо до вашої теми підходить ідея stream processing (коли ж постійний потік вхідних даних), то зробити PoC (proof of concept) для stream processing. Можна використати інструменти на ваш розсуд. Щоб полегшити вибір - почніть розгляд із apache flink або ksqldb.

Виконання

1. Варіанти використання event-driven архітектури у стрімінговому сервісі

У рамках побудови стрімінгового сервісу музики та подкастів, окремі функціональні сценарії потребують реалізації подієво-орієнтованої (event-driven) архітектури, що дозволяє ефективно реагувати на внутрішні зміни в системі та обробляти події в асинхронному режимі. Розглянемо основні варіанти використання такого підходу.

1.1. Відтворення музичного або подкаст-контенту

Події: track_played, track_paused, track_skippe

Видавець: сервіс Streaming

Підписники: сервіси Analytics, Recommendations, Ads

Опис: У процесі взаємодії користувача з аудіоконтентом формуються події про відтворення, які надсилаються до брокера повідомлень. Ці події використовуються аналітичним сервісом для обліку статистики, сервісом рекомендацій — для персоналізації, сервісом реклами — для оптимізації контенту.

1.2. Проведення оплати користувачем

Подія: payment_successful

Видавець: сервіс Payments

Підписники: сервіси Analytics, Emails, Users

Опис: Після успішної транзакції формується подія, яка активує оновлення облікового запису користувача, формування фінансової статистики, а також надсилання підтвердження електронною поштою.

1.3. Реєстрація нового користувача або оновлення профілю

Події: user_registered, user_updated

Видавець: сервіс Users

Підписники: сервіси Emails, Analytics, Recommendations

Опис: Події формуються при створенні нового профілю або зміні існуючих даних. Вони використовуються для запуску розсилок, збору статистичних даних та побудови початкового профілю користувача для рекомендацій.

1.4. Створення або редагування плейлистів

Події: playlist_created, playlist_updated, playlist_deleted

Видавець: сервіс Playlists

Підписники: сервіси Analytics, Recommendations

Опис: Інформація про взаємодію користувачів з плейлистами є ключовою для формування вподобань, оцінки популярності композицій та адаптації рекомендацій.

1.5. Реакція на рекомендації

Події: track_liked, track_disliked

Видавець: сервіс Streaming

Підписник: сервіс Recommendations

Опис: Результати взаємодії користувача з рекомендованим контентом передаються у вигляді подій, які використовуються для динамічної адаптації алгоритмів персоналізації.

1.6. Обробка помилок або відмов системи

Події: stream_failed, payment_failed

Видавець: відповідні сервіси (Streaming, Payments)

Підписники: сервіси Analytics, Alerts

Опис: Такі події фіксують помилки в роботі системи й можуть бути використані для аналітики, а також генерації сповіщень адміністраторам.

1.7. Побудова аналітики в реальному часі

Події: агреговані події від Streaming, Users, Payments

Підписник: сервіс Real-Time Analytics (на базі Apache Flink або ksqlDB)

Опис: Поточні події надходять до системи обробки потоків для формування актуальних дашбордів, рейтингу треків, моніторингу активності користувачів.

2. Патерни подієво-орієнтованої архітектури, що використовуються у системі

У запропонованій архітектурі стрімінгового сервісу використовуються кілька основних патернів подієво-орієнтованого

програмування. Їх застосування забезпечує масштабованість, асинхронність, відмовостійкість і розширюваність системи.

2.1. Патерн "Publish/Subscribe" (публікація-підписка)

Опис: Даний патерн дозволяє сервісам надсилати події (publish), не знаючи, хто їх отримає, а інші сервіси — підписуватись (subscribe) на потрібні їм події. Взаємодія реалізується через брокер повідомлень (наприклад, Apache Kafka).

Приклад використання у системі: Сервіс Streaming публікує події track_played, які споживаються сервісами Analytics та Recommendations.

2.2. Патерн "Event Notification" (сповіщення про події)

Опис: Сервіс надсилає подію з мінімальною інформацією, лише вказуючи, що певна дія відбулась. Підписані сервіси можуть самостійно отримати деталі, звернувшись до джерела.

Приклад використання у системі: Сервіс Users після успішної реєстрації надсилає подію user_registered, після чого Emails Service ініціює надсилання привітального повідомлення.

2.3. Патерн "Event-Carried State Transfer" (подія з передачею стану)

Опис: Подія несе у собі повну або часткову інформацію про стан об'єкта. Це дозволяє споживачу не звертатись до джерела події, а обробити все необхідне самостійно.

Приклад використання у системі: Сервіс Payments надсилає подію payment_successful, яка містить дані про користувача, суму, час транзакції — для зберігання в Analytics Service.

2.4. Патерн "Event Sourcing"

Опис: Замість збереження лише поточного стану, зберігається вся послідовність змін у вигляді подій. Це забезпечує повну відтворюваність стану.

Можливість використання: У разі потреби відтворення історії дій користувача або розвитку об'єкта (наприклад, історія редагувань плейлистів) можна застосувати цей підхід. Поточна реалізація допускає розширення системи в цьому напрямку.

2.5. Патерн "CQRS" (Command Query Responsibility Segregation)

Опис: Відокремлення команд (запису) та запитів (читання) у різні сервіси або шари. Події формуються після виконання команд і слугують тригером для оновлення проєкцій (read-models).

Приклад використання у системі: Після створення плейлиста (playlist_created), подія ініціює оновлення read-моделі в аналітичному сервісі або у швидкому кеші рекомендацій.

2.6. Патерн "Competing Consumers" (конкурентне споживання)

Опис: Кілька інстансів одного сервісу одночасно підписані на події з одного топіку й обробляють їх паралельно, забезпечуючи масштабування обробки.

Приклад використання у системі: Analytics Service може масштабуватись у кілька екземплярів для обробки великого обсягу подій track_played.

3. Інструменти та технології, що використовуються у подієво-орієнтованій архітектурі

Для реалізації подієво-орієнтованої архітектури у стрімінговому сервісі було обрано сучасний стек інструментів, що забезпечують надійність, масштабованість та легкість інтеграції між мікросервісами. Нижче наведено основні з них із обґрунтуванням вибору.

3.1. Apache Kafka

Тип: брокер повідомлень

Призначення:

- передача подій між сервісами у режимі publish/subscribe;
- зберігання історії подій (log-based streaming);
- можливість масштабування.

Обґрунтування вибору: Kafka — промисловий стандарт для побудови високонавантажених розподілених систем із асинхронною обробкою. Завдяки надійності, високій пропускну здатності та підтримці stream processing Kafka ідеально підходить для подієвої взаємодії сервісів, зокрема у стрімінговому застосунку.

3.2. ksqlDB або Apache Flink

Тип: платформи обробки потоків даних

Призначення:

- обробка подій у реальному часі;
- побудова агрегатів, фільтрація, трансформації;
- збереження агрегованих результатів у базах даних або кешах.

Обґрунтування вибору: Ці інструменти дозволяють реалізувати stream processing без необхідності створення складної логіки вручну.

- ksqlDB — працює поверх Kafka, дозволяє писати обробку подій на SQL-подібній мові;
- Flink — гнучкіший варіант для більш складних сценаріїв, зокрема з використанням Java/Scala/Python.

3.3. NestJS + Kafka Module (або інший Kafka-клієнт)

Тип: серверний фреймворк

Призначення:

- створення мікросервісів;
- інтеграція з Kafka через спеціалізовані модулі;
- реалізація логіки обробки подій.

Обґрунтування вибору: NestJS добре підтримує мікросервісну архітектуру і надає вбудовані механізми для роботи з Kafka. Це дозволяє швидко і надійно реалізувати продюсерів і консюмерів подій.

3.4. Docker + Docker Compose

Тип: система контейнеризації

Призначення:

- розгортання Kafka, ZooKeeper, ksqlDB/Flink, сервісів тощо;
- забезпечення локального або продакшн-середовища.

Обґрунтування вибору: Контейнеризація забезпечує портативність, швидке налаштування середовища та простоту розгортання. Docker Compose дозволяє швидко підняти цілу екосистему Kafka та супутніх сервісів для тестування та розробки.

3.5. PostgreSQL + Redis

Призначення:

- PostgreSQL: основна СУБД для зберігання постійних даних сервісів;
- Redis: кеш для швидкої обробки запитів або збереження агрегованих результатів обробки подій.

Обґрунтування вибору: Redis використовується для тимчасового зберігання аналітичних даних або рекомендацій, які обчислюються в реальному часі. PostgreSQL залишається основною базою для кожного мікросервісу у межах патерну "DB per Service"

4. Діаграма

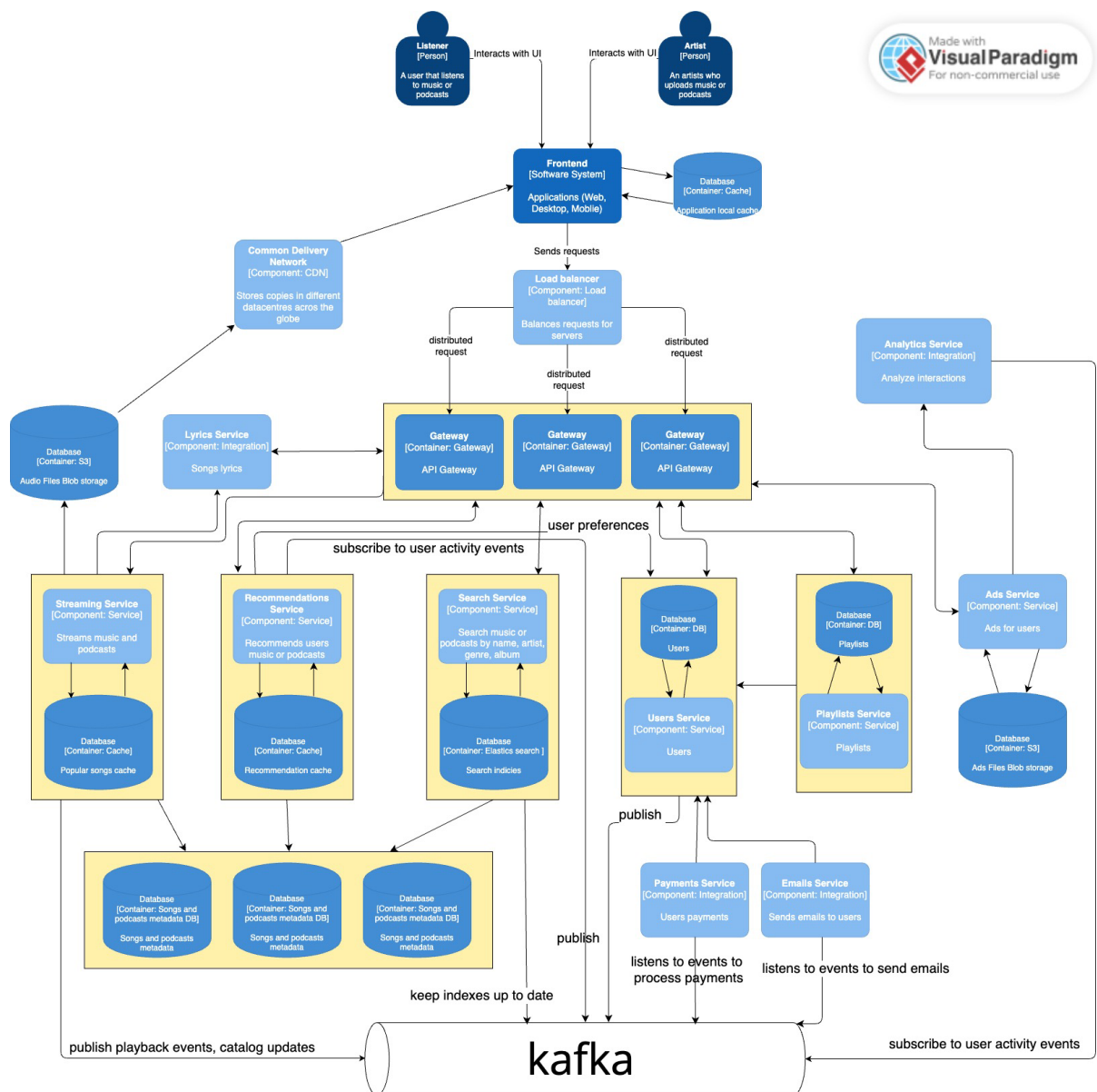


Рис. 1 – Оновлена діаграма

5. PoC

З метою демонстрації працездатності подієво-орієнтованої архітектури було реалізовано прототип (PoC) взаємодії між двома мікросервісами у рамках сценарію **відтворення музичного контенту**. При цьому використовувались технології Kafka як брокера подій, NestJS як основна серверна платформа для мікросервісів, та Kafka UI — для моніторингу подій.

5.1. Сценарій

Після запуску треку користувачем сервіс **streaming-service** формує подію `track_played`, яка надсилається до Kafka. У свою чергу, **analytics-service** підписується на відповідний топик і реагує на подію — наприклад, обчислює статистику або зберігає дані для подальшого аналізу.

5.2. Технічна реалізація

- **Streaming-service** реалізований на базі NestJS. Він містить HTTP-контролер, який приймає запит `POST /play` і надсилає подію `track_played` до Kafka.
- **Analytics-service** реалізовано як NestJS microservice, що використовує `@nestjs/microservices` для підписки на Kafka-топик `track_played`.
- Усі сервіси працюють локально, а Kafka запускається у Docker-контейнері, доступному через доменне ім'я `kafka.apps.orb.local:9092`.
- Для зручного перегляду подій і топиків використано **Kafka UI**, розгорнуту як окремий сервіс через `docker-compose`.

5.3. Результати

- Події `track_played` успішно надсилаються та приймаються.

- Мікросервіси не взаємодіють напряму, комунікація здійснюється виключно через Kafka.
- Kafka UI дозволяє візуально відстежити оброблені події, структуру топіків та активність consumer-груп.
- Рішення довело можливість масштабованої, асинхронної, слабо зв'язаної взаємодії сервісів у рамках обраної архітектури.

Код можна знайти в репозиторії:

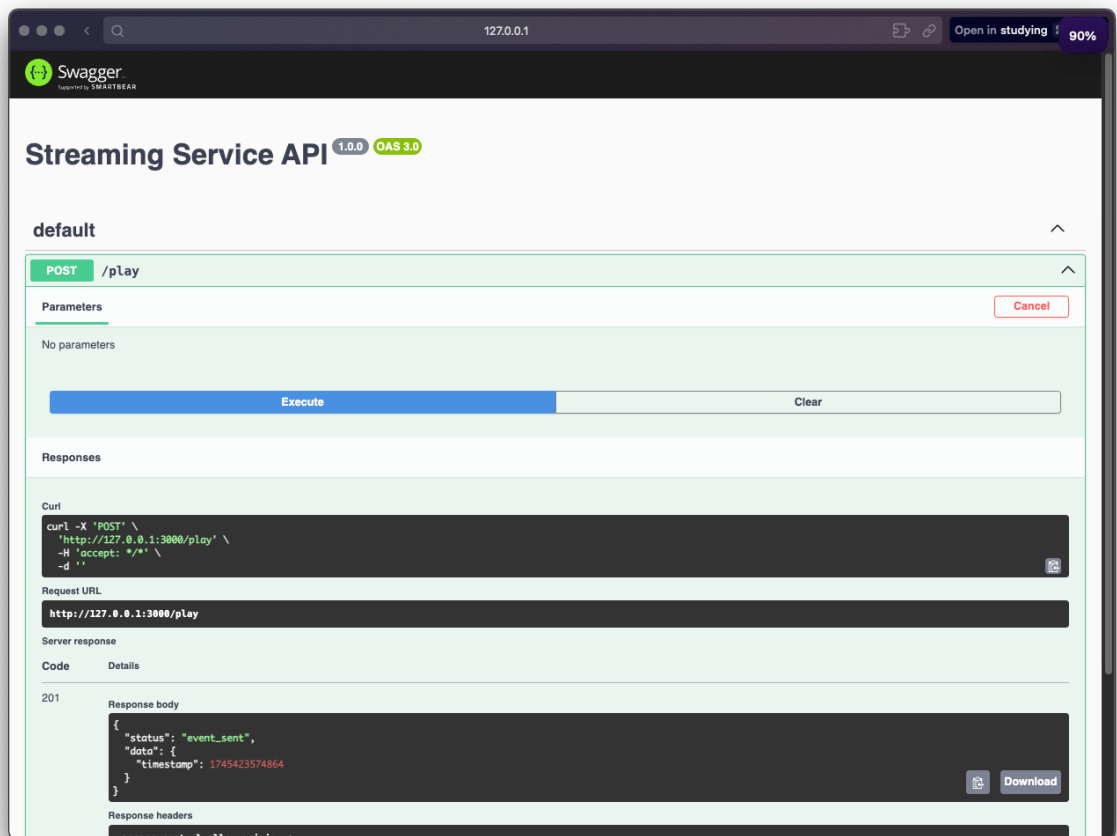


Рис. 1 – Streaming Service

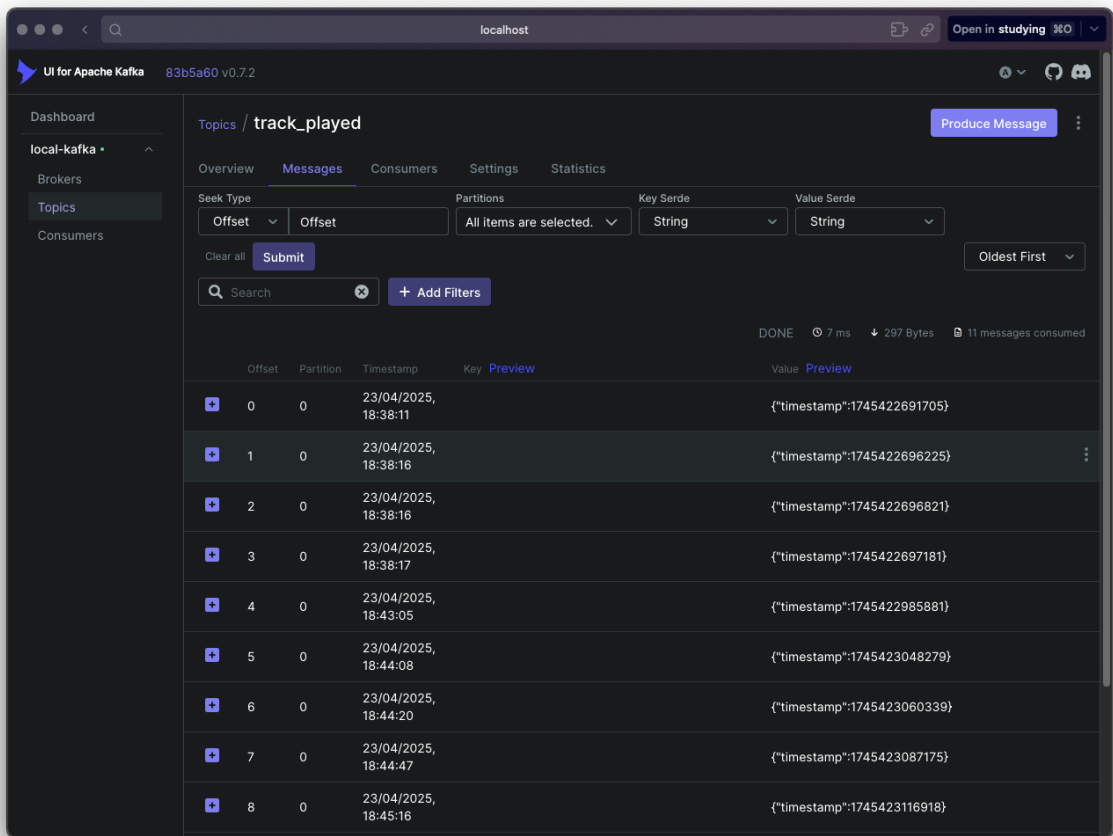


Рис. 2 – Візуалізація Kafka

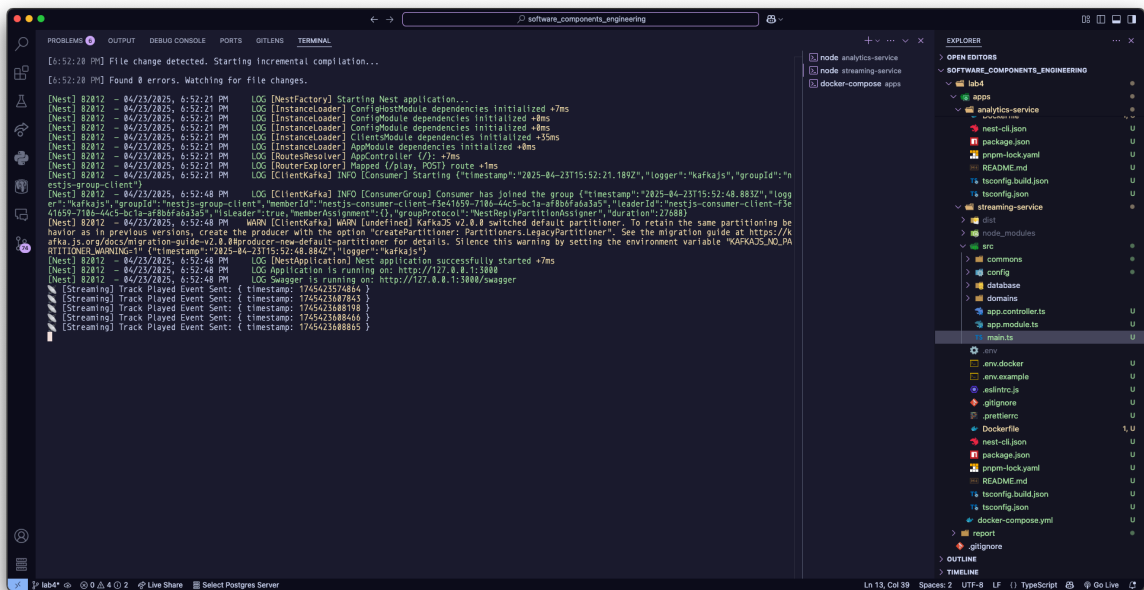


Рис. 3 – Запуск Streaming-service

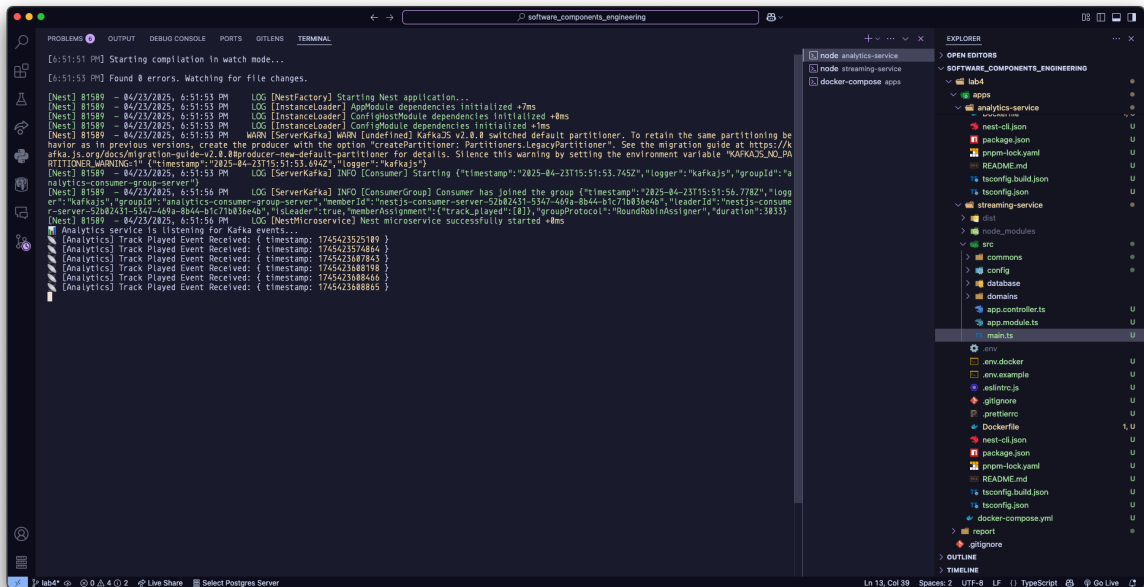


Рис. 4 – Запуск Analytics-service

Висновок:

У результаті виконання лабораторної роботи було реалізовано подієво-орієнтовану архітектуру для стрімінгового сервісу музики та подкастів. Було проаналізовано функціональні сценарії, які доцільно реалізовувати за допомогою подій, зокрема взаємодію користувача з контентом, здійснення оплати, оновлення профілю та інші.

Для реалізації взаємодії між мікросервісами використано брокер повідомлень Apache Kafka. Було застосовано основні архітектурні патерни подієвої взаємодії, зокрема Publish/Subscribe, Event Notification, Event-Carried State Transfer, CQRS та Competing Consumers. Сервіси реалізовано на основі фреймворку NestJS з підтримкою асинхронної обробки подій через Kafka.

У рамках PoC реалізовано два мікросервіси: streaming-service, який надсилає події track_played, та analytics-service, який підписується на ці події та обробляє їх. Для візуалізації подій використано Kafka UI. В результаті тестування було підтверджено працездатність подієвої взаємодії

між сервісами, а також здатність архітектури до масштабування та модульності.

Отримані результати підтверджують доцільність застосування event-driven підходу для побудови сучасних розподілених систем, де важливо забезпечити гнучкість, відмовостійкість та можливість подальшого розширення функціональності без порушення роботи існуючих компонентів.